

CAPSTONE PROJECT (AI and MLOps) PROJECT PROPOSAL – COHORT 4 GROUP 3

1. Project Title

Fine-Tuning Made Simple (FineTunelt): A Platform for Personalizing Language Models on Your Own Data

2. Brief Problem Statement

With large language models (LLMs) becoming mainstream, many organizations or individuals want to leverage fine-tuned models OR/ AND create Small Language Models (SLMs) for specific tasks but may lack in LLM performance on specific tasks with other adaptation techniques such as Prompt Engineering & RAG and may not have the technical expertise, computational resources, or infrastructure to effectively fine-tune models.

With ***FineTunelt***, we aim to create an accessible, user-friendly platform that democratizes fine-tuning large language models (LLMs) as easy and accessible as possible. With this platform, even users who aren't machine learning experts can upload their own dataset, choose a model, set basic preferences, and get a custom-trained LLM in return. The service handles everything—from spinning up cloud GPUs, to fine-tuning the model using PEFT, and uploading the final version to the user's HuggingFace account for download and inference.

3. Background Information

Domain Information

Fine-tuning LLMs has become an essential technique for adapting pre-trained models to specific domains, tasks, or styles. However, the process may involve significant challenges:

- Availability & Preparation of high-quality relevant training data
- Pre-Trained Model (PTM) selection & model performance evaluation across multiple dimensions
- Selecting appropriate hyperparameters
- Managing computational resources efficiently
- Deploying and monitoring models in production environments

These challenges present substantial barriers to entry for many potential users who could benefit from customized AI models.

Problem Description and Analysis

Current fine-tuning solutions typically fall into two categories:

- **DIY approaches:** Requiring significant ML expertise, infrastructure management, and technical knowledge
- **Limited-scope services:** Offering narrow capabilities, such as simple parameter-tuning and may have complex and challenging data preparation for specific tasks or deployment options.

The gap from these approaches pose challenges in domain adaptation of pre-trained GPTs OR creation of SLMs for specific tasks. Organizations often need to either invest heavily in ML talent or settle for suboptimal out-of-the-box solutions that don't meet their specific requirements. We aim to address some key challenges such as:

- Complexity in data-preparation and curation for fine-tuning
- Lack of guidance on optimal fine-tuning parameters for specific use cases
- Challenges in deploying and monitoring fine-tuned models
- Cost inefficiencies in resource allocation during training

Possible Applications

FineTuneHub would serve numerous application scenarios:

1. **Domain-specific models:** Healthcare, legal, financial, and other specialized industries
2. **Enterprise-specific knowledge assistants:** Companies can fine-tune models on their internal documentation
3. **Content generation with brand voice:** Marketing teams creating content with consistent tone
4. **Multilingual adaptations:** Enhancing model performance for specific languages
5. **Research accelerators:** Allowing researchers to experiment with fine-tuning without infrastructure hurdles

4. Motivation for Selection of the Project

Our motivation stems from several key observations in the current AI landscape:

- **Democratization gap:** While base models are increasingly accessible, the ability to customize them remains limited to technical specialists
- **Resource inefficiency:** Organizations often waste computational resources due to suboptimal fine-tuning practices

- **MLOps complexity:** The full lifecycle management of fine-tuned models requires sophisticated infrastructure. By addressing these challenges through a unified platform, we aim to significantly lower the barrier to entry for custom AI model development while demonstrating the full spectrum of MLOps best practices.

5. Detailed Dataset Description and Dataset Source

This platform is designed for users to bring their own data. The data is expected to be provided by the end user in a HF dataset format. It should be packaged in a format that HuggingFace's datasets library can read—this means a ZIP file with train, validation, and optionally test splits. We will provide some samples and guidance to create the dataset in the required format.

We will support several use-cases, such as:

- Instruction-following tasks (like Alpaca-style prompts and responses)
- Conversational fine-tuning
- Reasoning based Question Answering

Our backend will check the structure of the uploaded data and give users a preview, so they know their dataset is good to go before training starts.

Some example Dataset which we intend to use:

- 1) **Stanford Alpaca:** A dataset of 52,000 instruction-following demonstrations generated using OpenAI's text-davinci-003 engine.

Source: [GitHub - tatsu-lab/stanford_alpaca](https://github.com/tatsu-lab/stanford_alpaca)

Format: JSON format with instruction-input-output triplets

Size: ~80MB

License: Creative Commons Attribution (CC BY)

- 2) **Anthropic's Helpful and Harmful dataset (HH-RLHF)**

Source: [Hugging Face](https://huggingface.co/datasets/Anthropic/hh-rlhf)

Format: Paired helpful/harmful responses

Size: ~500MB

Usage: For evaluation and demonstration of safety fine-tuning

- 3) **OpenAssistant Conversations Dataset (OASST1)**

Source: [Hugging Face](https://huggingface.co/datasets/OpenAssistant/oasst1)

Format: Conversational data with human feedback

Size: ~5GB

Usage: For conversational fine-tuning examples

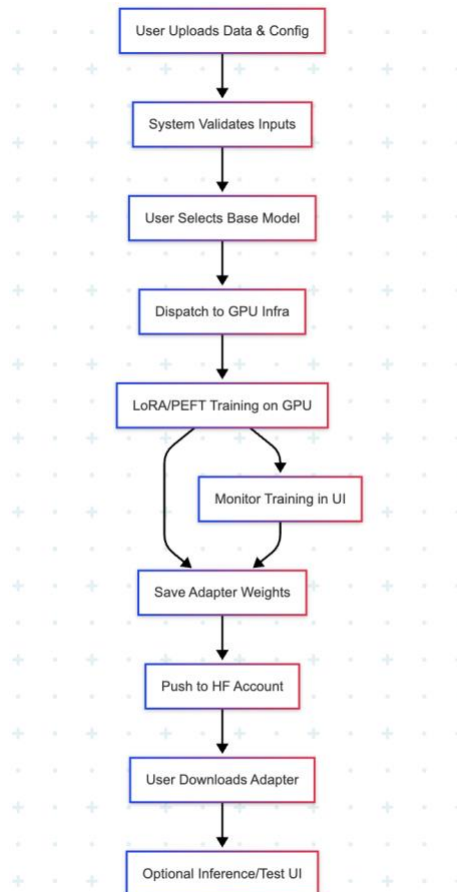
6. Current Benchmark

The user can run the benchmarks specific to the domain they have. As an example if it's a maths dataset, maths specific benchmarks can be run to evaluate the finetuning. If domain is about code, then code specific benchmarks have to be run by the end user. As a service we cannot provide benchmarks for all possible domains, so this is the end user's responsibility. Several existing solutions provide capabilities in the fine-tuning space but present certain complexities as listed below.

- **OpenAI Fine-tuning API:** Limited to OpenAI models, Minimal guidance on data preparation [OpenAI Fine-tuning Documentation](#)
- **Google Cloud Vertex AI Custom Training:** Enterprise-focused solution with high technical barrier [Google Vertex AI Documentation](#)
- **Scale AI Fine-tuning Platform:** Enterprise solution with emphasis on data preparation [Scale AI Fine-tuning](#)
- **Research Benchmarks:** LoRA, QLoRA, and PEFT techniques (Hu et al., 2021)

7. Proposed Plan

Below is a simple high level preliminary architecture flow of the components involved in fine-tuning as a service.



a. **Approach: Our platform will implement a comprehensive workflow.**

- We will use the HF TRL library for applying Supervised finetuning (SFT) once a link to the HF dataset is available. Alternates could be PPO or GRPO (deep seek methodology) for RL based rewards.
- Control of certain hyperparameters like learning rate, optimizer etc will be provided to the end user.
- End user can choose to pick the defaults.
- The job runs as a background. The system provides feedback via an API as to where in processing the job is.
- Once completed the model can be uploaded automatically to HF and the user can download and consume it via the standard HF transformers library.
- We propose to use Qwen 3b as the pretrained base model as this is a general purpose model with good performance in maths and coding tasks, so we expect this to adapt to reasoning better.

1. Data Ops Pipeline

- Automated data validation (guardrails, dataset size) and quality assessment
- Guided data formatting and structure recommendation

2. Fine-Tuning Orchestration

- Automated hyperparameter recommendation based on task type and dataset
- Support for multiple fine-tuning techniques (LoRA, QLoRA, full fine-tuning)
- Resource optimization based on model size and data characteristics
- Checkpointing and automatic recovery

3. Deployment and Monitoring

- One-click model push to HuggingFace
- Performance monitoring
- Version management and rollback capabilities
- Usage analytics

b. **Packages and Tools**

We intend to use some memory optimizations via frameworks like Unsloth and a 3r party GPU infrastructure like RunPod or any other, to do the finetuning on demand (more like a serverless way). Our base model would be Qwen 3b , we can work with either Ampere series (A10 or A100 or RTX). This can also be programmed based on availability of these specific GPUs at a certain time and region.

Development Tools:

- Framework: Unsloth for training abstraction
- Model Libraries: Hugging Face Transformers, PEFT
- LLM-Ops: KubeFlow/ MLflow for experiment tracking, Weights & Biases for visualization
- Data Processing: Pandas, NLTK, SpaCy
- Evaluation: BERTScore, ROUGE, Perplexity score, custom metrics
- Infrastructure: Docker for Containerization, RunPod OR/ AND Kubernetes for orchestration, CI/CD: GitHub Actions

Core Technologies:

- Python 3.10+
- FastAPI for backend services
- ReactJS for frontend interface
- PostgreSQL for metadata/ data storage
- Redis for caching and job queues
- HF for storage

c. Algorithms**Parameter-Efficient Fine-Tuning (PEFT) techniques:**

- LoRA (Low-Rank Adaptation) OR QLoRA (Quantized Low-Rank Adaptation)

Output will be an adapter which either can be separately uploaded or merged with the base model.

Optimization algorithms:

- AdamW optimizer with learning rate scheduling Mixed precision training

d. Metrics**Training Metrics:**

- Loss curves (training and validation)
- Learning rate progression
- Gradient norms
- Memory usage efficiency

Performance Metrics:

- Task-specific scores (e.g., accuracy, F1)
- ROUGE scores for generation tasks
- BERTScore for semantic similarity Inference latency and throughput
- Perplexity score

Project Stages and Deliverables**1. Platform Architecture and Data Pipeline (Week 1-2) Deliverables:**

- System architecture document
- Data validation and preprocessing pipeline
- Initial UI wireframes

2. Fine-Tuning Engine Development (Week 2-5) Deliverables:

- Working fine-tuning pipeline with LoRA/QLoRA support
- Explore Hyperparameter optimization module
- Training monitoring dashboard

3. Evaluation Framework (Week 5-6) Deliverables:

- Multi-dimensional evaluation system
- Comparative visualization tools
- Performance analytics dashboard

4. Deployment System and UI (Week 6-7) Deliverables:

- Publishing model on hugging face infrastructure / Object store
- Complete user interface
- Documentation and usage examples

Deployment Plan

We will create a full-stack application with the following components:

1. Fine-Tuning Service Frontend:

- React-based UI with Material UI components
- Deployed on Cloud Infrastructure

2. **Backend API:** FastAPI application
 - Containerized with Docker
 - Deployed on AWS ECS/ Serverless or HF
3. **Model Serving for Proof-of-Concept with Inference:**
 - RunPod based Inference Endpoints for production deployment
4. **Proof-of-Concept Demo Interface for Inference:**
 - Gradio/ React based UI

MLOps Implementation

1. **Experiment Tracking:**
 - MLflow for logging all experiments / Run Pod
 - Integration with Weights & Biases for advanced visualizations
2. **CI/CD Pipeline:**
 - GitHub Actions for automated testing
 - Container registry integration
 - Automated deployments
3. **Monitoring:**
 - Capturing system metrics (Prometheus, etc.)
 - Model Statistics (W&B)
4. **Version Control:**
 - Git for code
 - DVC for dataset versioning
 - Model registry for versioned models

8. Preliminary Exploratory Data Analysis

Since we are building a service that will fine-tune a base model based on the user-input data (governed by the HF Dataset format), we may not be able to carry in-depth EDA. However, we intend to perform EDA on the datasets we would use to build the training and inference pipelines. Below is a sample of how an EDA might look e.g. of the Stanford Alpaca dataset and associated insights:

1. **Distribution of instruction types:**
 - x% open-ended generation
 - y% information extraction
 - z% classification/categorization
 - p% rewriting/reformatting
 - q% other specialized tasks
2. **Input length distribution:**
 - Mean: ## tokens
 - Median: ## tokens
 - Maximum: ## tokens
 - Shows right/ left skewed distribution with majority under tokens
3. **Output quality assessment:**
 - x% of responses rated as complete and responsive
 - y% contain minor omissions or inconsistencies
 - z% show significant quality issues
4. **Thematic coverage:**
 - Good coverage of general knowledge topics
 - Limited representation of specialized domains Underrepresentation of multilingual content

Sample of Key insights from this analysis:

- The dataset provides good coverage for general instruction tuning
- Augmentation with domain-specific data will be necessary
- Quality filtering should be implemented to remove problematic examples
- We should consider upsampling underrepresented instruction types

For the fine-tuning platform, we intend to also explore implementation of an automated/ assisted EDA pipeline that:

1. **Generates Standard Reports**
 - Basic statistics on dataset composition

- Warning flags for potential quality issues
- Recommendations for data augmentation or filtering
- 2. Provides Interactive Exploration**
 - Sample browser with filtering capabilities
 - Faceted search across different dimensions
 - Comparison view between original and fine-tuned model outputs
- 3. Offers Predictive Insights**
 - Identifying examples that might cause overfitting
 - Predicting potential problematic examples for the fine-tuning process
 - Recommending optimal dataset size based on task complexity

9. Expected Outcomes

Upon completion, FineTuneHub will deliver:

1. Domain Adapted Fine-tuned Model
2. Multi Tenant Service
3. A comparison of the Inference from Base Model & Domain Adapted Model

1. End-to-end Language fine-tuning platform with:

Intuitive UI for non-technical users
Comprehensive documentation and tutorials

2. Knowledge artifacts:

Our Learnings & Observations for various fine-tuning scenarios

10. Project Demonstration Strategy

We plan to demonstrate our project through several formats:

1. Live Interactive Demo:

Web-based interface showing the complete workflow
Real-time fine-tuning of a small model on curated dataset
Comparative evaluation of before/after performance

2. Use Case Demonstrations:

Domain adaptation
Methodologies such as use-cases across Q&A, Conversational & Reasoning

3. Technical Architecture Walkthrough:

System component interactions
Scaling capabilities
Security and data privacy considerations

4. Results Visualization:

Before/after comparisons on key metrics
Training efficiency : Analytics Resource utilization statistics

5. Future Roadmap Presentation:

Expansion possibilities
Integration opportunities with existing tools
Research directions for further improvement

11. Proposed Timeline of Project Stage Executions

Phase 1 (May 19-June 2)

Complete system architecture design
Set up development environment and CI/CD pipeline
Implement data validation pipeline

Phase 1 (May 26 - June 9)

Develop data preprocessing and validation tools
Create initial UI wireframes
Set up MLflow experiment tracking

Phase 2 (June 2- June 30)

Implement LoRA/QLoRA fine-tuning engine
Develop hyperparameter recommendation system
Create training monitoring dashboard

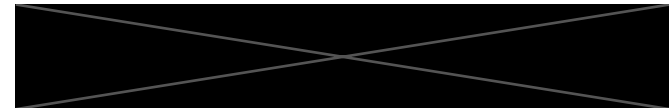
- Mentored Session 1 Goals:** Present architecture and data pipeline
- Mentored Session 2 Goals:** Demo data processing pipeline with quality metrics
- Mentored Session 3 Goals:** Show working fine-tuning process on sample dataset

Phase 3 (June 23 - Jul 7)

- Develop deployment system
- Create model serving infrastructure
- Implement monitoring and alerts Complete main UI components
- Mentored Session 4 Goals:** Demonstrate end-to-end system with initial results

Phase 4 (June 30-July 10)

- System integration and testing
- Documentation and tutorials
- Performance optimization
- Prepare final demonstration



12. Team Members' Names

Group 3			
Group 3			
Group 3			
Group 3			
Group 3			
Group 3			
Group 3			
Group 3			
Group 3			
Group 3			
Group 3	Rakesh Rallapalli		rallapalli.rakesh@gmail.com

13. Designated Team Coordinator's Name

Group 3			
Group 3			
Group 3	Rakesh Rallapalli		rallapalli.rakesh@gmail.com